CSE 276A – Homework 1 Report

Brenton Dunn (TSN 200220892, bmdunn@ucsd.edu)
Allen Xu (TSN 200480686, alx006@ucsd.edu)
Demo video: https://youtu.be/qkejUP5_e7c

Algorithm Description:

Our robot traverses the waypoint list with an open-loop so it relies on a calibrated model of how the robot moves. The model rests on four assumptions. Each motion type (forward, spin left, spin right) uses fixed motor power, and only the duration changes. The battery level and floor surface stay constant. The robot cannot slide sideways, so it only drives straight or pivots in place. We also note that distance and rotation are proportional to time.

load_waypoints converts each line of waypoints.txt into a Waypoint object with x,y,θ data, with validation checks. For each waypoint the robot first rotates to face it, then drives straight for the time needed to cover the distance, and then pivots to the rotation requested by the waypoint if a change is needed.

Kinematic Model and Control Algorithm:

The kinematic model is a differential drive robot with hardset power levels for forward and rotation movements. The position (x, y, θ) is integrated from the commanded motion. It is implemented as EstimatedPose.integrate and updated on every iteration of the roughly 20 Hz control loop:

$x \mathrel{+}= v \cdot \cos(\theta) \cdot dt$
$y \mathrel{+}= v \cdot \sin(\theta) \cdot dt$
$\theta = \text{normalize}(\theta + \omega \cdot dt)$

Here v is the forward speed and ω the turn rate. Both are zero except during the motion that is active. The normalize helper wraps angles so the robot always takes the shortest turn. (350° = -10°)

The robot has three motions, each with a fixed wheel command and a calibrated speed:

Motion	Wheel command (L, R)	Calibrated v / ω
Forward	(0.25, 0.20)	v = 0.298 m/s
Spin left	(-0.45, 0.30)	ω = +1.59 rad/s
Spin right	(0.45, -0.30)	ω = −1.83 rad/s

The forward command is unequal because of a lean discovered during calibrations. The two spin directions have separate rates due to wheel variance.

The controller in **velocity_control.run_route** works in timed segments. To turn toward a target, it computes the rotational difference between the target and the estimated heading. The sign of the difference picks the spin direction, and the robot spins for difference/rotation rate seconds. This repeats until the estimated heading is within 0.01 rad of the target. To drive, it commands the forward power for distance/velocity seconds, and until the estimated position is within 1 cm of the waypoint. Every segment is capped at 4s, and a 0.3 s pause follows each segment so the robot settles before the next one. During testing we found that distance travelled did not scale linearly with time even though power was constant. For example, 4 seconds of driving at constant power was not 2x longer than driving 2 seconds. Because of this, we only travelled in intervals which we calibrated for.

Code:

The code is in the **robot_control** package. **kinematic_control.py** holds the calibration constants, normalize_angle and the EstimatedPose class, which together make up the kinematic model. **velocity_control.py** holds the Waypoint class, the waypoint loader, the timed turn, drive and pause logic in run_route, and the ROS node entry point. The controller publishes left right wheel power levels to motor_commands to actually move the wheels.

Calibration:

Calibration started with straight-line driving. We started with equal power to both wheels. As we observed drift, we added and subtracted power from one side in a binary search-like methodology until the robot consistently moved in a straight line. This gave the final forward command of L = 0.25 and R = 0.20. We then timed a straight run with those powers. It covered 23.5 in in 2 s, so v = 0.298 m/s.

For rotation, we pivoted the robot in place and measured how its body moved for a starting point. After tuning the pivot powers, we timed a 90° turn in each direction, which took 0.989 s to the left and 0.859 s to the right.

Performance:

Straight-line motion was consistent, and the power calibration held well. Rotation was the main source of error. The wheels have high friction and sometimes slip unpredictably, so the same command produces different rotation amounts. The pivot axis also changed between turns. Sometimes the robot turned about its center, and other times about the left or right wheels, which shifts its position slightly with each turn. These small errors compounded along the route. The calibration was also sensitive to motor power and battery state. Powers that were too low or too high changed the rotation a lot. Charging the robot for even 15 minutes changed the power profile enough to make our previous calibration unusable, so we had to recalibrate.

Because the controller is open-loop, nothing corrects these errors. Here are our observations of the robot in real world conditions:

Waypoint	Error
(1,0)	~1 in right of target (right-side pivot)
(1,1)	~1 in right (previous misalignment, rotation shifted back a bit)
(2,1)	~2 in right (previous misalignment)
(2,2)	~4 in off (right-side pivot shifted the robot up during the 180° rotation)
(1,1)	~6 in off
(0,0)	~12 in off